

RegEx এডভান্সড মাস্টার গাইড (পার্ট ২)

বাস্তব প্রজেক্ট, কমপ্লেক্স প্যাটার্ন ও ডাটা ক্লিনআপ গাইড

এই পর্বে আপনি কী কী শিখবেন?

প্রথম পর্বে আপনি RegEx-এর মৌলিক উপাদান ও কোয়ান্টিফায়ার শিখেছেন। এই পার্ট ২-এ আমরা জানব গ্রুপিং (Grouping), লুক-অ্যাহেড (Lookahead), ডাটা রিপ্লেসমেন্ট (Search & Replace) এবং বাস্তব প্রজেক্টে ব্যবহৃত জটিল কিছু প্যাটার্ন!

১. এডভান্সড কনসেপ্ট (গ্রুপিং ও ব্যাকরেফারেন্স)

কনসেপ্ট / চিহ্ন	কাজ	উদাহরণ
<code>(...)</code> (Capturing Group)	নির্দিষ্ট অংশকে ব্রাকেটে ঘিরে গ্রুপ করা এবং মেমোরিতে ধরে রাখা।	<code>(\d{3})-(\d{4})</code> (কোড ও নম্বর আলাদা করা)
<code>(?: ...)</code> (Non-capturing Group)	গ্রুপ করবে কিন্তু মেমোরিতে সেভ করবে না (পারফরম্যান্স ফাস্ট করার জন্য)।	<code>(?:http https)://</code>
<code>,</code> (Backreference)	আগে গ্রুপে পাওয়া টেক্সটকে পুনরায় মেলানো।	<code>(\w+)\s+\1</code> (ডাবল হয়ে যাওয়া শব্দ যেমন "the the" ধরা)
<code>(?= ...)</code> (Positive Lookahead)	সামনে নির্দিষ্ট কিছু থাকলে তবেই ম্যাচ করবে (কিন্তু সেটিকে সিলেক্ট করবে না)।	<code>\d+(?=\\$)</code> (ডলার চিহ্নের আগে থাকা সংখ্যা ধরা)

২. বাস্তব জীবনের আরও ৫টি বাস্তব উদাহরণ

উদাহরণ ৫: URL / ওয়েবসাইট লিঙ্ক এক্সট্রাক্ট করা

লক্ষ্য: যেকোনো HTTP বা HTTPS ওয়েবসাইট লিঙ্ক টেক্সট থেকে খুঁজে বের করা।

RegEx প্যাটার্ন: `https?://[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}(\?/\S*)?`

প্যাটার্নের ব্যাখ্যা:

- `https?://` - `http://` অথবা `https://` মেলানো (এখানে 's' ঐচ্ছিক)।
- `[a-zA-Z0-9.-]+` - ডোমেইন নেম (যেমন: github বা google)।
- `\.[a-zA-Z]{2,}` - ডট এবং অন্তত ২ অক্ষরের টিএলডি (যেমন: .com, .org, .edu)।
- `(\?/\S*)?` - ইউআরএল-এর পেছনের পথ (যদি থাকে)।

উদাহরণ ৬: IP Address (IPv4) শনাক্ত করা

লক্ষ্য: নেটওয়ার্কের IP Address (যেমন: 192.168.1.1) ফিল্টার করা।

RegEx প্যাটার্ন: `(?:\d{1,3}\.){3}\d{1,3}`

প্যাটার্নের ব্যাখ্যা:

- `\d{1,3}\.` - ১ থেকে ৩ ডিজিটের সংখ্যা এবং একটি ডট (যেমন: 192.)।
- `{3}` - এই ডট সহ সংখ্যার অংশটি পর পর ৩ বার রিপিট হবে (192.168.1.)।
- `\d{1,3}` - শেষে শুধু ১ থেকে ৩ ডিজিটের একটি সংখ্যা থাকবে (1)।

উদাহরণ ৭: HTML Tag এক্সট্রাকশন

লক্ষ্য: HTML কোডের ভেতরে থাকা ট্যাগগুলো ধরে ফেলা বা মুছে ফেলা (যেমন: `<h1>Header</h1>`)।

RegEx প্যাটার্ন: `<[>]+>`

প্যাটার্নের ব্যাখ্যা:

- `<` - শুরু হবে `<` চিহ্ন দিয়ে।
- `[>]+` - `>` চিহ্ন না পাওয়া পর্যন্ত যেকোনো ক্যারেক্টার এক বা একাধিকবার থাকবে।
- `>` - শেষ হবে `>` চিহ্ন দিয়ে।

উদাহরণ ৮: সোশ্যাল মিডিয়া হ্যাশট্যাগ (#Hashtag) এবং মেশন (@Mention)

লক্ষ্য: পোস্টের ভেতর থেকে #bangladesh বা @username ফিল্টার করা।

RegEx প্যাটার্ন (Hashtag): `#\w+`

RegEx প্যাটার্ন (Mention): `@\w+`

উদাহরণ ৯: ডুপ্লিকেট শব্দ (Repeated Words) ধরা

লক্ষ্য: ভুলবশত পর পর দুবার লেখা একই শব্দ (যেমন: "This is **is** a test") খুঁজে বের করা।

RegEx প্যাটার্ন: `(\w+)\s+`

প্যাটার্নের ব্যাখ্যা:

- `(\w+)` - প্রথম শব্দটি ১ নম্বর গ্রুপে ধরে মেমোরিতে রাখা হলো।
- `\s+` - মাঝে এক বা একাধিক স্পেস।
- ঠিক প্রথম গ্রুপে পাওয়া শব্দটিই আবার রিপিট হলে ম্যাচ করবে।

৩. টেক্সট রিপ্লেসমেন্ট (Search & Replace Magic)

VS Code বা যেকোনো টেক্সট এডিটর বা প্রোগ্রামিং ল্যান্সুয়েজে RegEx দিয়ে শুধু সার্চ নয়, লেখার ফরম্যাট রূপান্তরও করা যায়:

```
[ সার্চ টেক্সট ] : 01712-345678 (ফোন নম্বর)
[ RegEx সার্চ ] : (\d{5})-(\d{6})
[ Replace Pattern]: +880 $1 $2
[ আউটপুট টেক্সট ] : +880 01712 345678
```

৪. প্রোগ্রামিং ল্যান্সুয়েজে RegEx ব্যবহারের কোড

Python উদাহরণ:

```
import re

text = "যোগাযোগ করুন: info@example.com অথবা 01700000000"
emails = re.findall(r'[\w.-]+@[ \w.-]+\.\w+', text)
phones = re.findall(r'01[3-9]\d{8}', text)
```

```
print("ইমেইল:", emails) # ['info@example.com']  
print("ফোন:", phones)  # ['01700000000']
```

JavaScript উদাহরণ:

```
const text = "অর্ডার আইডি: #ORD-99823";  
const pattern = /#ORD-\d+/g;  
const match = text.match(pattern);  
  
console.log(match); // ["#ORD-99823"]
```